A Trimester – 1 Class Project Report on

Weather and Air Quality Monitoring Dashboard

Submitted in the partial fulfillment of the requirements for

26CS5101E – Computational Thinking and Programming

Master of Technology In

Artificial Intelligence & Data Science

By

Student ID	Student Name
2601150060	Shaik Bhikku Saheb

under the supervision of

Dr. G. Syam Prasad

Professor

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd., u/s 3 of UGC Act 1956)

Greenfields, Vaddeswaram, Guntur (Dist.), Andhra Pradesh - 522502

2026 – 27

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the Trimester – 1 Class Project Report entitled "**Weather and Air Quality Monitoring Dashboard** " is being submitted by Shaik Bhikku Saheb and 2601150060 submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University is a record of Bonafide work carried out under our guidance and supervision during the academic year 2026 – 2027.

Dr. G. Syam Prasad

Project Supervisor

Signature

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DECLARATION

The Trimester -1 Class Project Report entitled "**Weather and Air Quality Monitoring Dashboard**" is a record of Bonafide work of Shaik Bhikku Saheb and 2601150060 submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University.

Shaik Bhikku Saheb – 2601150060

ACKNOWLEDGEMENT

It is great pleasure for me to express my gratitude to our honorable President **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the trimester - 1 class project report.

I express sincere gratitude to HOD of AI&DS Dept. **Dr. M. Venkata Naresh** for his leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank for providing us the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. G. Syam Prasad** for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted themselves directly or indirectly to make this project report success.

ABSTRACT

The Weather and Air Quality Monitoring Dashboard is a compact web application designed to present weather conditions, short-term forecasts and air-pollution measurements in one place. Python performs server-side processing and API communication, while a single HTML file contains the interface structure, CSS styling and JavaScript interactions. A lightweight Flask server serves the dashboard and provides a JSON endpoint to the browser.

Users can search for a city and view available measurements such as temperature, feels-like temperature, humidity, wind speed, weather conditions, forecast highs and lows, Air Quality Index (AQI), PM2.5 and PM10. Additional pollutant values are shown only when supplied by the data provider. The browser updates dashboard cards and charts using the structured response returned by Python.

The project demonstrates problem decomposition, API-based retrieval, dictionaries and lists, reusable functions, input validation, exception handling, data transformation, browser-server communication and software testing. The application is intended for academic demonstration; it is not an official environmental monitoring service and does not provide medical advice. Results depend on API coverage, update frequency and the AQI scale selected.

Keywords: Python, Flask, HTML, CSS, JavaScript, weather forecast, air quality, AQI, API integration, dashboard, software testing.

INDEX

S. No	TITLE	Page Numbers
1	Introduction	7
2	Objectives and Requirements	8
3	System Architecture	9
4	Workflow and Algorithms	10
5	Component 1:Problem Solving& Computational Thinking	11-12
6	Component 2: Programming & Data Structures	13-14
7	Component 3: Software Design & Engineering	15-16
8	Component 4: Functional Programming & Data Handling	17-18
9	Component 5: Software Quality & Testing	19-20
10	Component 6: AI-Assisted Software Engineering	21
11	Execution, Sample Output & Technical Defence	22-23
12	Conclusion, Future Scope & References	24

1. INTRODUCTION

Problem Statement

Weather and air-pollution information is often presented by separate services. Users may need to check several pages to understand current conditions, forecast changes and pollution levels for a location. The project addresses this by combining available weather and air-quality information in one simple dashboard.

Motivation

This domain offers a practical way to apply computational thinking. It requires input validation, external data retrieval, structured response processing, error handling and presentation of numerical information. The implementation is compact enough to explain during a project defence and does not require a large frontend framework.

Proposed Solution

A Flask application serves dashboard.html and exposes a JSON endpoint. Python validates the requested city, retrieves coordinates, calls the weather and air-quality services, and combines their responses. JavaScript updates the page with current readings, forecast entries, charts and error messages.

Assumptions

- Internet connectivity is required for live data retrieval.
- The chosen API may not provide every field for every location.
- API units and AQI scales must be explicitly labelled.
- The application will not invent substitute values when data is missing.

2. OBJECTIVES AND REQUIREMENTS

Objectives

- Combine current weather and air-quality information in one interface.
- Allow users to search for a city by name.
- Display current conditions and a multi-day forecast when available.
- Show AQI, PM2.5, PM10 and other supported pollutant values.
- Use Python functions to retrieve, validate and normalize API responses.
- Present data using a single-page HTML, CSS and JavaScript dashboard.
- Handle invalid input, network errors, timeouts and missing fields clearly.
- Test core functions and failure scenarios before demonstration.

Functional Requirements

ID	Requirement
FR1	Accept and validate a city name.
FR2	Resolve the city to coordinates using geocoding.
FR3	Retrieve current weather and forecast information.
FR4	Retrieve available air-quality and pollutant values.
FR5	Display readings, forecast cards and charts.
FR6	Refresh the dashboard after another city search.
FR7	Display helpful messages when a location or service is unavailable.

Non-Functional Requirements

Readability, modularity, request timeouts, safe error handling, responsive layout, clear units, privacy-conscious design and testability are the main non-functional requirements.

3.SYSTEM ARCHITECTURE

The application uses a lightweight request-response architecture. The browser handles user interaction and display, Flask serves the HTML page and API endpoint, and Python functions communicate with external data providers.

```
USER / BROWSER
      |
      v
dashboard.html
(HTML + CSS + JavaScript)
      |
      | GET /api/dashboard?city=...
      v
FLASK APPLICATION (weather_dashboard.py)
  | Validate input
  | Geocode city
  | Fetch weather data
  | Fetch air-quality data
  | Normalize response
      |
      v
JSON RESPONSE
      |
      v
Dashboard cards, forecast and charts
```

Main Modules

Module	Responsibility
Browser interface	Search form, data cards, charts, loading and error states.
Flask route	Serves the page and provides the JSON endpoint.
Geocoding function	Converts city text to coordinates and a display name.
Weather function	Requests current conditions and daily forecast fields.
Air-quality function	Requests supported AQI and pollutant fields.
Normalization layer	Creates a predictable data structure for JavaScript.
Tests	Checks validation, transformations and error handling.

External API Endpoints

Service	Purpose	Base endpoint
Geocoding	City to coordinates	geocoding-api.open-meteo.com/v1/search
Weather	Current and forecast data	api.open-meteo.com/v1/forecast
Air quality	AQI and pollutant data	air-quality-api.open-meteo.com/v1/air-quality

Verify current API parameters, field availability, attribution requirements and usage limits against the provider documentation during implementation

WORKFLOW AND ALGORITHMS

Dashboard Workflow

1. The user enters a city name and submits the search form.
2. JavaScript shows a loading indicator and requests the local Flask endpoint.
3. Python trims and validates the city input.
4. The geocoding service returns coordinates and a location label.
5. Python requests weather and air-quality information for those coordinates.
6. The application checks response status and handles missing fields.
7. Python returns a normalized JSON response.
8. JavaScript updates cards, forecast information and charts.
9. If a request fails, the dashboard displays a readable error message.

Reservation-Style Critical Steps Adapted to Data Retrieval

```
START
Read city name
Validate city name
Find latitude and longitude
Request weather and air-quality data
IF a required service fails:
    return a clear error response
ELSE:
    normalize available values
    return dashboard JSON
Render data in the browser
END
```

Data Processing Rules

- Use the resolved latitude and longitude for weather and air-quality requests.
- Label temperature, wind speed and pollutant concentration with provider-defined units.
- Select and label one AQI scale; do not mix US AQI and European AQI.
- Represent missing measurements as N/A, not zero.
- Show location and local time context where available.

COMPONENT 1: PROBLEM SOLVING & COMPUTATIONAL THINKING

Decomposition and Abstraction

The problem is divided into city lookup, weather retrieval, air-quality retrieval, response normalization and browser rendering. A location is represented by a display name and geographic coordinates. Weather and air-quality responses are treated as structured records rather than displayed as raw JSON.

Algorithm Selection

Geocoding is delegated to a specialized service. After coordinates are found, the application retrieves weather and air-quality data and converts them into a stable response shape. No complex optimization algorithm is needed because the main task is retrieval and presentation.

Complexity and Cost Analysis

Operation	Local complexity / cost	Explanation
Validate city	O(C)	C is the length of the city string.
Geocoding request	Network-dependent	Latency depends on provider response.
Weather request	Network-dependent	Depends on requested fields and network.
Air-quality request	Network-dependent	Coverage and available fields vary.
Normalize response	O(F)	F is the number of processed fields.
Render forecast	O(D)	D is the number of forecast entries.

Network latency normally dominates local processing time. Explicit timeouts, clear loading feedback and avoidance of unnecessary requests improve the user experience.

COMPONENT 1.1: ALGORITHM CODE & ANALYSIS

Input Validation

```
def validate_city(city: str) -> str:
    cleaned = city.strip()
    if not cleaned:
        raise ValueError("Enter a city name.")
    if len(cleaned) > 100:
        raise ValueError("City name is too long.")
    return cleaned
```

Safe API Retrieval Pattern

```
def request_json(url, params):
    response = requests.get(url, params=params, timeout=10)
    response.raise_for_status()
    data = response.json()
    if not isinstance(data, dict):
        raise ValueError("Unexpected API response.")
    return data
```

Missing-Data Strategy

External services may return partial responses or omit a pollutant at a particular location. The application should check fields before using them and preserve missing values as unavailable. Substituting zero for a missing AQI or temperature would be misleading because zero is a valid measurement.

Algorithmic Judgement

The first version focuses on a single selected city. Caching, multi-city comparison, historical trends and scheduled alerts may be useful later, but are not necessary for demonstrating the core workflow.

COMPONENT 2: PROGRAMMING & DATA STRUCTURES

Selected Structures

Structure / technique	Purpose	Reason

Dictionary	Normalized API response	Named fields are easy to access and serialize.
List	Daily forecast entries	Maintains forecast order.
String	City names and labels	Represents user input and display text.
Float / integer	Weather and pollutant values	Represents numeric measurements.
Functions	Validation and API calls	Supports reuse and unit testing.
Exceptions	Input and service failures	Separates failures from normal results.

Python Techniques

- Type hints document expected inputs and return values.
- Dictionary access and explicit checks handle optional fields.
- Requests timeouts avoid waiting indefinitely for a service.
- Flask route handlers connect browser requests to Python logic.
- JSON serialization transfers structured data to JavaScript.
- Exception handling produces useful errors without exposing internal traces.

COMPONENT 2.1: PYTHON CODE DESIGN

API Configuration and Data Normalization

```
GEOCODING_URL = "https://geocoding-api.open-meteo.com/v1/search"
WEATHER_URL = "https://api.open-meteo.com/v1/forecast"
AIR_QUALITY_URL = "https://air-quality-api.open-meteo.com/v1/air-quality"
```

```
def safe_value(mapping: dict, key: str):
    value = mapping.get(key)
    return value if value is not None else None

dashboard_data = {
    "location": location_name,
    "temperature_c": safe_value(current, "temperature_2m"),
    "humidity_percent": safe_value(current, "relative_humidity_2m"),
    "wind_speed_kmh": safe_value(current, "wind_speed_10m"),
    "aqi": safe_value(air_current, "us_aqi"),
    "pm2_5": safe_value(air_current, "pm2_5"),
    "pm10": safe_value(air_current, "pm10"),
}
```

Python Application Responsibilities

The Python file should contain the Flask application, city validation, geocoding, weather retrieval, air-quality retrieval, response normalization and route handlers. The browser endpoint should return only the fields needed by the interface.

Persistence

The initial project does not need a database because it displays current and forecast data from external services. Historical records and saved preferences can be added in a later version if required.

COMPONENT 3: SOFTWARE DESIGN & ENGINEERING

Separation of Responsibilities

The browser handles user interaction and visual updates. Flask validates incoming requests and coordinates the data retrieval. Dedicated Python functions perform geocoding, weather retrieval, air-quality retrieval and normalization. This separation keeps the code easier to test and explain.

Design Principles

- Single responsibility: each function performs one clear task.
- Encapsulation: API request details remain in Python functions.
- Consistent response shape: JavaScript receives predictable field names.
- Input validation: reject empty or unreasonable city input before API calls.
- Defensive programming: treat missing fields and service failures as expected cases.
- Configuration: keep endpoint URLs and timeout values in constants.

Route Design

Route	Method	Purpose	Response
/	GET	Serve dashboard page	HTML document
/api/dashboard?city=...	GET	Fetch combined city data	JSON or error

COMPONENT 3.1: SERVICE DESIGN & LIMITATIONS

Example Service Structure

```
def geocode_city(city: str) -> dict:
    # Request city matches and validate the result.
    ...

def fetch_weather(latitude: float, longitude: float) -> dict:
    # Request current conditions and forecast.
    ...
```

```
def fetch_air_quality(latitude: float, longitude: float) -> dict:
    # Request AQI and supported pollutant fields.
    ...

def build_dashboard(city: str) -> dict:
    # Validate, geocode, fetch and normalize data.
   ...
```

Why a Class Hierarchy Is Not Required

The initial project has a small number of service operations and no complicated domain hierarchy. Plain functions and dictionaries are sufficient and easier to understand. A service class could be introduced if the project later adds caching, multiple providers or configurable data sources.

Known Limitations

- The application depends on internet access and third-party API availability.
- City names may be ambiguous; the selected result should include region or country where available.
- Modelled air-quality values may differ from nearby physical sensor readings.
- The initial version does not store historical data or user preferences.
- A local Flask development server is not intended for production hosting.

COMPONENT 4: FUNCTIONAL PROGRAMMING & DATA HANDLING

Functional-Style Techniques

Small transformation functions, list construction and built-in operations can extract forecast values and prepare chart-ready data. These techniques are used where they improve clarity; unnecessary abstraction is avoided.

Data Pipeline

```
Raw API JSON
     |
     v
Validate response structure
     |
     v
Extract current and forecast fields
     |
     v
Normalize units and missing values
     |
     v
Build dashboard JSON
     |
     v
JavaScript creates cards and chart series
```

Forecast Transformation Example

```
def make_daily_forecast(daily: dict) -> list[dict]:
    dates = daily.get("time", [])
    highs = daily.get("temperature_2m_max", [])
    lows = daily.get("temperature_2m_min", [])
    codes = daily.get("weather_code", [])
    forecast = []
    for i, date in enumerate(dates):
        forecast.append({
            "date": date,
            "high_c": highs[i] if i < len(highs) else None,
            "low_c": lows[i] if i < len(lows) else None,
            "weather_code": codes[i] if i < len(codes) else None,
        })
    return forecast
```

COMPONENT 4.1: BROWSER DATA RENDERING

JavaScript Request Pattern

```
async function loadDashboard(city) {
  showLoading(true);
  clearError();

  try {
    const response = await fetch(
      `/api/dashboard?city=${encodeURIComponent(city)}`
    );
    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.error || "Unable to load dashboard.");
    }

    renderCurrentWeather(data.weather.current);
    renderAirQuality(data.air_quality.current);
    renderForecast(data.weather.daily);
    renderCharts(data);
  } catch (error) {
    showError(error.message || "Unexpected error.");
  } finally {
    showLoading(false);
  }
}
```

Concurrency and Asynchronous Behaviour

The initial Flask route can make requests sequentially for clarity. Weather and air-quality requests are independent after geocoding, so concurrent requests could reduce latency in a later version. Concurrency is not required for the first demonstration.

Units, AQI and Accessibility

- Use provider-documented units for all measurements.
- Choose and label one AQI scale, such as US AQI when requested and available.
- Use both category text and colour to communicate air-quality status.
- Give search controls visible labels and show loading/error messages.
- Use safe text insertion for API-provided strings rather than inserting untrusted HTML.

COMPONENT 5: SOFTWARE QUALITY & TESTING

Test Strategy

Test level	Test case	Expected result
Unit	Trim a valid city name	Clean city name returned
Negative	Empty city input	Validation error
Unit	Geocoding has no matches	Location-not-found response
Unit	Pollutant field missing	Value shown as unavailable
Integration	Valid city request	Combined dashboard JSON
Negative	API timeout	Useful error; server remains responsive
UI	Submit city search	Loading state then updated cards
UI	Unknown city	Readable error message

Quality Tools

pytest can execute automated tests. Ruff can lint and format Python files. These tools are recommended practices and should be reported as executed only after running them.

Suggested Commands

```
python -m pip install flask requests pytest
python weather_dashboard.py
python -m pytest -q
python -m pip install ruff
ruff check weather_dashboard.py test_dashboard.py
ruff format --check weather_dashboard.py test_dashboard.py
```

COMPONENT 5.1: TEST CODE & VALIDATION

Example Unit Tests

```
import pytest
from weather_dashboard import validate_city

def test_validate_city_trims_spaces():
    assert validate_city("  Vijayawada  ") == "Vijayawada"

def test_validate_city_rejects_empty_input():
    with pytest.raises(ValueError):
        validate_city("   ")

def test_validate_city_rejects_long_input():
    with pytest.raises(ValueError):
        validate_city("x" * 101)
```

Acceptance Criteria

- The dashboard opens from the local Flask server.
- A valid city returns weather and air-quality data when provider coverage exists.
- Invalid input produces a clear message.
- Unavailable measurements are not fabricated or treated as zero.
- The browser remains usable after a service error.
- Automated tests pass in the environment used for the demonstration.

CI/CD and Deployment

A simple CI pipeline can install dependencies and run tests and lint checks on each change. For public deployment, use a production WSGI server, disable Flask debug mode, review provider terms and apply request limits.

COMPONENT 6: AI-ASSISTED SOFTWARE ENGINEERING

Specification-First Development

AI tools may assist with HTML layout, API request code, chart rendering and test drafting. Before accepting generated code, define the expected fields, units, error cases and acceptance criteria. AI-generated code is a draft, not proof of correctness.

Example Prompt

"Create an educational Weather and Air Quality Dashboard using Python 3.10+, Flask, Requests and one HTML file containing HTML, CSS and JavaScript. Use Open-Meteo geocoding, weather forecast and air-quality services. Include city search, current weather, a seven-day forecast, AQI and supported pollutant values, clear units, request timeouts, validation, safe error handling and pytest tests. Do not invent data or claim tests passed unless they were run."

Review Checklist

- Verify endpoint URLs and parameter names against current documentation.
- Confirm that units and AQI scale are correctly labelled.
- Test unknown cities, missing fields and API timeouts.
- Check that HTTP status codes and request timeouts are handled.
- Review browser rendering for unsafe HTML insertion.
- Run the application and tests after significant changes.
- Disclose AI assistance according to course requirements.

Responsible Use

Do not provide credentials or unnecessary personal information to external AI tools. The student remains responsible for understanding the code, checking its behaviour, respecting API terms and following academic-integrity requirements.

EXECUTION, SAMPLE OUTPUT & TECHNICAL DEFENCE

Project Folder

```
Weather_Air_Quality_Dashboard/
|-- weather_dashboard.py
|-- dashboard.html
|-- test_dashboard.py
|-- requirements.txt   (optional)
```

Installation and Execution

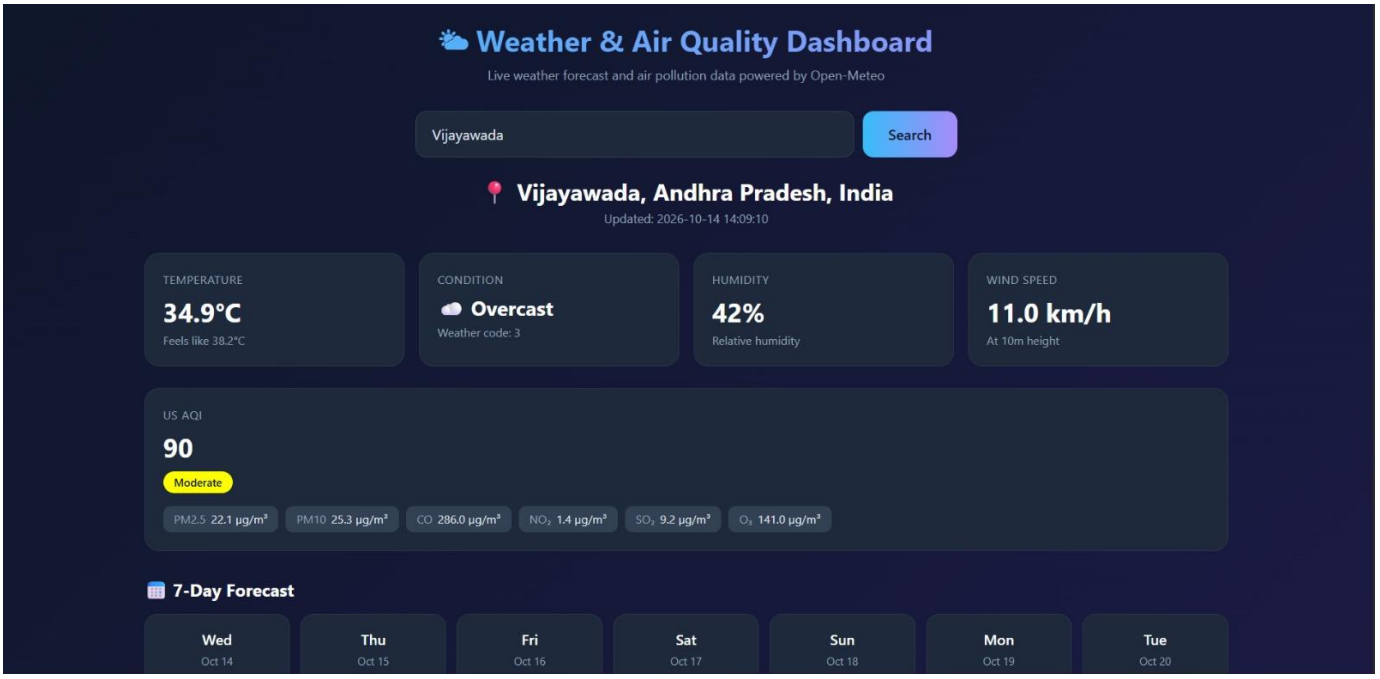
```
python -m pip install flask requests
python weather_dashboard.py
```

Open the local URL printed by Flask, typically http://127.0.0.1:5000, in a browser. Keep the terminal open while using the dashboard. The port may differ if the application configuration changes.

Illustrative Dashboard Output

Dashboard area	Intended display
Location	City, region and country when available
Current weather	Temperature, feels-like, humidity, wind and conditions
Forecast	Daily minimum/maximum temperature and weather description
Air quality	AQI when available, PM2.5, PM10 and supported pollutants
Status	Loading indicator, updated time or readable error

This table describes the planned interface, not a live API response. Actual values depend on the selected location, time and provider. Screenshots and execution results should be added only after running the completed implementation.


Weather & Air Quality Dashboard
Live weather forecast and air pollution data powered by Open-Meteo
Vijayawada
Search
Vijayawada, Andhra Pradesh, India
Updated: 2026-10-14 14:09:10
TEMPERATURE
34.9°C
Feels like 38.2°C
CONDITION
Overcast
Weather code: 3
HUMIDITY
42%
Relative humidity
WIND SPEED
11.0 km/h
At 10m height
US AQI
90
Moderate
PM2.5 22.1 µg/m³
PM10 25.3 µg/m³
CO 286.0 µg/m³
NO₂ 1.4 µg/m³
SO₂ 9.2 µg/m³
O₃ 141.0 µg/m³
7-Day Forecast
Wed
Oct 14
Thu
Oct 15
Fri
Oct 16
Sat
Oct 17
Sun
Oct 18
Mon
Oct 19
Tue
Oct 20

CONCLUSION, FUTURE SCOPE & REFERENCES

Conclusion

The Weather and Air Quality Monitoring Dashboard demonstrates how a practical information problem can be decomposed into a compact Python web application. It combines API integration, input validation, structured data processing, a single-page HTML interface, JavaScript rendering and software testing. The design is suitable for explaining computational-thinking and software-engineering concepts during a project defence.

Future Scope

- Add caching to reduce repeated API requests.
- Allow comparisons between saved cities.
- Store historical readings after selecting an appropriate data-retention approach.
- Add optional browser location detection with explicit user permission.
- Improve forecast visualizations and accessibility.
- Add a secondary data provider only after verifying compatible fields and terms.
- Deploy with a production WSGI server, monitoring and request limits.

References

1. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
2. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/
3. Requests Documentation. https://requests.readthedocs.io/
4. Open-Meteo. Weather Forecast API Documentation. https://open-meteo.com/en/docs
5. Open-Meteo. Air Quality API Documentation. https://open-meteo.com/en/docs/air-quality-api
6. Open-Meteo. Geocoding API Documentation. https://open-meteo.com/en/docs/geocoding-api
7. pytest Documentation. https://docs.pytest.org/

Implementation Note

The code samples in this report are illustrative excerpts. The complete runnable implementation must define build_dashboard and all browser rendering functions, and the project must be executed and tested before reporting